

Dědičnost

Karel Richta a kol.

Katedra technických studií
Vysoká škola polytechnická Jihlava

© Karel Richta, 2021

Objektově-orientované programování, OOP
02/2021, Lekce 4

<https://moodle.vspj.cz/course/view.php?id=201479>



Dědičnost

Jak dosáhneme opakované použitelnosti kódu:

C

- Funkce a jejich volání na různých místech programu
- Vytváří se hlavičkové soubory, kde jsou deklarace funkcí

C++

- Zapouzdřením dat do tříd odstraníme duplicity i pro proměnné
- Dědičnost - mechanismus tvorby - odvozování tříd z již existujících

Dědičnost podporuje abstrakci:

- Potomek dodržuje rozhraní od předka
- Potomek disciplinovaně mění chování předka – implementuje nové funkce a vlastnosti

Omezujeme tak duplicity v kódu, které:

- Omezují přehlednost kódu
- Zvyšují riziko chyb
- Zvyšují náročnost úprav (více míst)
- Zvyšují nároky na paměť

Dědičnost (pokr.)

Pracuje s mechanismem vztahu mezi třídami – rodič a potomek.

Základní vlastnosti:

- Potomek dědí vlastnosti a chování předka (rodiče) – členská data i metody.
- Potomek může metody upravovat, nebo překrývat (přepsat).

Standardní postup:

- Identifikace podobných tříd v modelované oblasti.
- Určení společných vlastností a chování (členská data a metody).
- Vytvoření nové třídy obsahující právě společné atributy.
- Nová tříd se stane rodičem předchozích tříd – ty se stanou potomky.
- V původních třídách již nejsou společné atributy explicitně určeny.

Příklad:

Třída: pes, kočka, člověk

Společné vlastnosti – jméno, stáří, pohlaví

Nová třída – rodič Savec se společnými vlastnostmi

Potomci mají navíc speciální vlastnosti – Pes (známka, barva_srsti)

Dědičnost vytváří hierarchii tříd (objektů) - generalizace a specializace.

Veřejná dědičnost

- Modeluje vztah „je“, „je druhu“, „is A“ (ISA-vztah).
- Pro úpravu vlastností tříd dědičností není nutné mít zdrojový kód, pouze prototypy.
- Odvození třídy potomek od rodiče (základní) je explicitně dáno v deklaraci potomka (v rodiči není nic uvedeno).
- Předpis: `class Potomek : public Rodic` – operátor :
- Příklad: `class Clovek : public Savec{}`
- Objekt odvozené třídy v sobě obsahuje objekt základní třídy.
- Veřejné položky rodiče se stanou veřejnými položkami potomka.
- Soukromé položky se stanou součástí potomka – přístupné pouze pomocí veřejných metod.
- Pokud vynecháte klíčové slovo **public** – je dědičnost automaticky **private** (soukromá dědičnost) – vše je soukromé.
- Položky potomka nejsou vůbec přístupné základní třídě – ta o nich neví.

Veřejná dědičnost (pokr.)

- Odvozená třída - nové položky, nové metody - nedostupné v rodiči (ten o nich neví).
- Odvozená třída musí mít svůj konstruktor.
- Při vytváření objektu potomka se vyvolá:
 - 1. konstruktor rodiče,
 - 2. konstruktor potomka.
- Konstruktor potomka předá informace konstruktoru rodiče.
- Destruktor se potomkovi nemusí přidávat, pokud není nutné něco dynamicky uvolňovat (vygeneruje se automaticky). Pokud ale potomek obsahuje odkazy na jiná data, pak se destruktory musí definovat.
- Pokud platnost objektu končí, volá se:
 - 1. destruktory potomka,
 - 2. destruktory rodiče.
- Potomek dědí všechny metody, konstruktor a destruktory je volán automaticky, nebo v seznamu inicializátorů.
- Nedědí se operátor přiřazení.

Veřejná dědičnost – konstruktor

- Konstruktor základní třídy:

```
Savec::Savec (string jmeno, int vek) {  
    this->m_jmeno = jmeno;  
    this->m_vek = vek;  
};
```

- Konstruktor odvozené třídy:

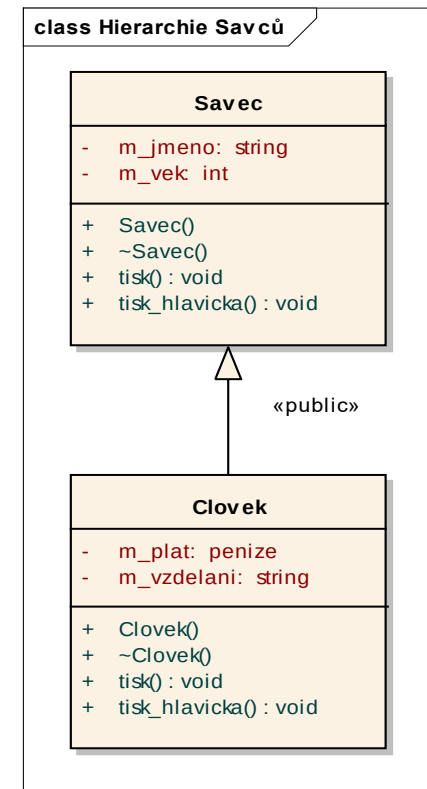
```
Clovek::Clovek(string str, int vek, int plat, string vzdelani) {  
    m_plat = plat;  
    m_vzdelani = vzdelani;  
    setVek(vek); // nelze: m_vek = vek  
    setJmeno(str) // lze, ale první se volá implicitní konstruktor  
                // základní třídy  
};  
Clovek::Clovek(string str, int vek, int plat, string vzdelani):  
    Savec(str,vek)    //seznam inicializátorů  
{  
    m_plat = plat;  
    m_vzdelani = vzdelani;  
};
```

Konstruktor odvozené třídy volá implicitní konstruktor základní třídy.

Lze uvést explicitně jaký konstruktor se bude volat – seznam inicializátorů (podle prototypu).

Pokud je explicitní konstruktor uveden , implicitní se již nebude volat.

Z konstruktoru odvozené třídy je vždy volán konstruktor základní třídy.



Inicializace objektu odvozené třídy objektem základní třídy

- Nový objekt vytvořen pomocí existujícího objektu základní třídy.
- Použit lze kopírovací konstruktor, pro vytvoření kopie objektu základní třídy – tím naplní odpovídající část odvozeného objektu.
- Existuje implicitní kopírovací konstruktor, lze použít pokud není nutné kopírovat dynamické objekty, nebo upravovat statická členská data.
- Konstruktor odvozené třídy:

```
Clovek::Clovek(Savec &sav, int plat, string vzdelani):Savec(sav) {  
    m_plat = plat;  
    m_vzdelani = vzdelani;  
};  
Savec::Savec(const Savec &sav) {  
    m_jmeno = sav.m_jmeno;  
    m_vek = sav.m_vek;  
    cout << "\nObjekt tridy Savec vytvoren (kop konstruktor): " << sav.m_jmeno;  
};
```

Použití rozhraní zákl. tříd ve třídách odvozených

- Nelze přímo přistupovat k členským datům základní třídy ve třídě (metodách) třídy odvozené.
- Musí se použít veřejných metod základní třídy – rozhraní (pokud jsou private).
- Pokud odvozená třída překryje metodu základní třídy, pak ji použije.
- Jinak bude používat metodu základní třídy.
- Pro exaktní určení metody je nutné použít operátor ::.

```
int Savec::get_vek() const {  
    return m_vek;  
};  
void Savec::tisk() const {  
    cout << "\nSavec jmeno: " <<m_jmeno;  
    cout << " vek: " <<m_vek;  
};
```

```
void Clovek::tisk_jmeno() const {  
    cout << get_jmeno();  
};  
void Clovek::tisk() const {  
    Savec::tisk();  
    cout<<" plat: "<<m_plat;  
    cout<<" vzdelani: "<<m_vzdelani;  
};
```


Použití

```
Savec s1;// existuje impl konstruktor  
Clovek c1;//existuje impl konstuktor  
s1.tisk();  
cout<<s1.get_vek() c1.tisk();  
cout<<c1.get_vek();
```

Savec s1;

m_jmeno

m_vek

Clovek c1;

m_jmeno

m_vek

m_plat

m_vzdelani

Statické proměnné a metody

- Jsou to data a metody třídy, nikoliv jednotlivých objektů.
- Jsou označena klíčovým slovem static.
- Statická data nejsou součástí žádné instance – musí se deklarovat odděleně.
- Statické metody mohou pracovat jen se statickými daty.
- Typický příklad: aktuální počet instancí třídy.

```
class Savec {  
private:  
    ...  
    static int pocet;  
public:  
    Savec (string, int);  
    Savec (const Savec &);  
    ~Savec();  
};
```

```
int Savec::pocet = 0;  
  
Savec::Savec (string jmeno, int vek) {  
    this->m_jmeno = jmeno; this->m_vek  
=vek;  
    pocet++;  
};  
  
Savec::~~Savec() { pocet --; };
```

Základní příklad dědičnosti – rodič Savec

```
class Savec {
private:
    string m_jmeno;
    int m_vek;
    static int pocet;
public:
    Savec (string, int);
    Savec (const Savec &);
    ~Savec();
    int get_vek();
    string get_jmeno();
    void tisk();
};
int Savec::get_vek() {
    return m_vek; };
string Savec::get_jmeno() {
    return m_jmeno; };
```

```
int Savec::pocet = 0;
Savec::Savec (string jmeno, int vek) {
    this->m_jmeno = jmeno; this->m_vek = vek;
    pocet++; cout <<"\nObjekt tridy Savec c. "
    << pocet<<"vytvoren : "<< m_jmeno;
};
Savec::~~Savec() { pocet --; cout <<"\npocet
objektu Savec: "<<pocet;
};
void Savec::tisk() {
    cout<<"\nSavec jmeno: "<<m_jmeno; cout <<"
    vek: "<< m_vek;
};
Savec::Savec(const Savec & sav) {
    m_jmeno = sav.m_jmeno; m_vek = sav.m_vek;
    pocet++; cout<<"\nObjekt tridy Savec c. " <<
    pocet
    <<"vytvoren (kop konstruktor):
    "<<sav.m_jmeno;
};
```

Základní příklad dědičnosti – potomek Člověk

```
class Clovek:public Savec {
private:
    int m_plat;
    string m_vzdelani;
    static int pocet;
public:
    void tisk();
    Clovek(string, int, int,
string);
    Clovek(Savec &, int, string);
    ~Clovek();
};
void Clovek::tisk() {
    Savec::tisk();
    cout <<" plat: " << m_plat <<
    " vzdelani: " << m_vzdelani;
};
```

```
int Clovek::pocet = 0;
Clovek::~~Clovek() {
    pocet --;
    cout <<"\npocet objektu Clovek: " << pocet;
};
Clovek::Clovek(string str, int vek, int plat, string
vzdelani):Savec(str,vek) {
    m_plat = plat; m_vzdelani = vzdelani;
    pocet++; cout<<"\nObjekt tridy Clovek c.
"<<pocet<<"vytvoren : " <<str;
};
Clovek::Clovek(Savec & sav,int plat, char*
vzdelani):Savec(sav) {
    m_plat = plat; m_vzdelani = vzdelani; pocet++;
    cout << "\nObjekt tridy Clovek c. " <<
    pocet <<"vytvoren: " << sav.get_jmeno();
};
```

Základní příklad dědičnosti - použití

```
int main() {  
    Savec sav1("Petr", 10);  
    Clovek oso1("Jan", 46, 10000, "VS");  
    Clovek oso2(sav1, 40000, "SS");  
    sav1.tisk();  
    oso1.tisk();  
    oso2.tisk();  
    return 0;  
}
```



```
C:\Windows\system32\cmd.exe  
Objekt tridy Savec c. 1vytvoren : Petr  
Objekt tridy Savec c. 2vytvoren : Jan  
Objekt tridy Clovek c. 1vytvoren : Jan  
Objekt tridy Savec c. 3vytvoren (kop konstruktor): Petr  
Objekt tridy Clovek c. 2vytvoren: Petr  
Savec jmeno: Petr vek: 10  
Savec jmeno: Jan vek: 46 plat: 10000 vzdelani: VS  
Savec jmeno: Petr vek: 10 plat: 40000 vzdelani: SS  
pocet objektu Clovek: 1  
pocet objektu Savec: 2  
pocet objektu Clovek: 0  
pocet objektu Savec: 1  
pocet objektu Savec: 0Pokračujte stisknutím libovolné klávesy... _
```

Chráněná členská data (protected)

- Občas potřebujeme přístup k členským datům rodičů v potomcích – použijeme mód **protected** - podobné **private**, ale pro členské metody potomků jako **public**.
- Přístupné jsou jen pomocí členských metod.
- Potomci ovšem mají přímý přístup ke chráněným položkám základní třídy.

```
//savec.h
class Savec {
private:
    string m_jmeno;
    int m_vek;
    static int pocet;
protected:
    char m_pohlavi;
};
```

```
class Clovek: public Savec {
    string m_vzdelani;
    int m_plat;
public:
    tisk();
}
void Clovek::tisk() {
    Savec::tisk();
    cout <<" plat: "<< m_plat;
    cout <<" vzdelani: "<< m_vzdelani;
    cout << "pohlavi :"<< m_pohlavi;
};
```

Chráněná členská data (protected)

- Občas potřebujeme přístup k členským datům rodičů v potomcích – použijeme mód **protected** - podobné **private**, ale pro členské metody potomků jako **public**.
- Přístupné jsou jen pomocí členských metod.
- Potomci ovšem mají přímý přístup ke chráněným položkám základní třídy.

```
//savec.h
class Savec {
private:
    string m_jmeno;
    int m_vek;
    static int pocet;
protected:
    char m_pohlavi;
};
```

```
class Clovek: public Savec {
    string m_vzdelani;
    int m_plat;
public:
    tisk();
}
void Clovek::tisk() {
    Savec::tisk();
    cout <<" plat: "<< m_plat;
    cout <<" vzdelani: "<< m_vzdelani;
    cout << "pohlavi :"<< m_pohlavi;
};
```

Přetypování v dědičnosti

Motivace: chceme jeden typ ukazatele a více objektů, na které ukazuje.

Standardně není povoleno přiřadit jednomu typu ukazatele ukazatel na jiný typ:

```
double x = 2.2;  
int * uk = & x; //nelze
```

Dědičnost – ukazatel na základní třídu může odkazovat i na objekt třídy odvozené.

Přetypování na předka:

- Nemusí se provést explicitní přetypování.
- Vše co může dělat objekt základní třídy může dělat i odvozená třída:
Clovek clo("Petr",40, 1, "VS");
Clovek * uk_clo = &clo;
Savec * uk_sav = &clo;
uk_sav->getJmeno(); //všechny metody budou funkční...

Přetypování na potomka:

- Musí být provedeno explicitně.
- Odvozená třída má nové metody a data, které u základní třídy nelze použít:
Savec sav("Ivan", 30);
Clovek *uk = (Clovek*)&sav;
cout<<uk->getPohlavi(); // pouze metody ze základní třídy budou funkční

Statická vazba

- Spuštění přetížené metody závisí na objektu, který ji vyvolal <= platí jen pro spuštění metody pomocí tečkové notace:

```
Savec sav; // Savec::tisk() {cout<<"Savec";}
Clovek clo; // Clovek::tisk(cout<<"Clovek");
sav.tisk();
clo.tisk();
```

- V případě spouštění pomocí ukazatelů je implicitně spuštěna metoda určená typem ukazatele – ne konkrétního objektu:

```
Savec * uk_sav;
uk_sav = &sav;
uk_sav->tisk();
uk_sav = &clo;
uk_sav->tisk(); // volá se Savec::tisk(), protože uk_sav je ukazatel na savce
                // napraví to dynamická vazba
```

Dynamická vazba

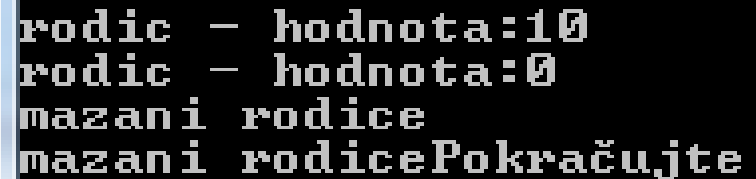
- Statická vazba – již v době překladač je jasné, která funkce bude spuštěna.
- Dynamická vazba – program zodpovídá za spuštění správné metody za běhu podle objektu který reprezentuje.
- Nástroj pro aktivaci dynamické vazby je virtuální členská metoda.
- V prototypu virtuální členské metody v základní třídě použijeme klíčové slovo **virtual**.
- Přetížené metody v základních třídách mohou překrýt danou funkci ze základní třídy – dle rozhodnutí programátora.
- Dynamická vazba musí udržovat informace o zpracovávaných objektech, statická nemusí – méně náročná na provedení.
- Pokud nepoužívám dědění – není nutná dynamická vazba.
- Pokud nechci metodu předefinovat v odvozené třídě – neuvedu virtual.
- Pokud není virtuální metoda v potomkovi vytvořena, bude použita metoda základní třídy.

Vzorový příklad statické vazby

```
class Rodic {
    int r;
public:
    Rodic(int a):r(a) {};
    Rodic() {r = 0;}
    ~Rodic() {cout<<"\nmazani rodice";}
    void tisk () {cout<<"\nrodic - hodnota:"<<r;}
};

class Potomek: public Rodic {
    int p;
public:
    Potomek(int a):p(a) {};
    ~Potomek() {cout<<"\nmazani potomka";}
    void tisk() {cout<<"\npotomek "
        "hodnota: "<< p;}
};
```

```
int main() {
    Rodic * pole[2];
    pole[0] = new Rodic(10);
    pole[1] = new Potomek(100);
    pole[0]->tisk();
    pole[1]->tisk();
    delete pole[0];
    delete pole[1];
    system("PAUSE");
    return 0;
}
```



```
rodic - hodnota:10
rodic - hodnota:0
mazani rodice
mazani rodicePokračujte
```

Vzorový příklad - dynamická vazba

```
class Rodic {
    int r;
public:
    Rodic(int a):r(a) {};
    Rodic() {r = 0;}
    virtual ~Rodic() {cout<<"\nmazani
rodice";}
    virtual void tisk () {cout<<"\nrodic - "
        "hodnota:"<<r;}
};

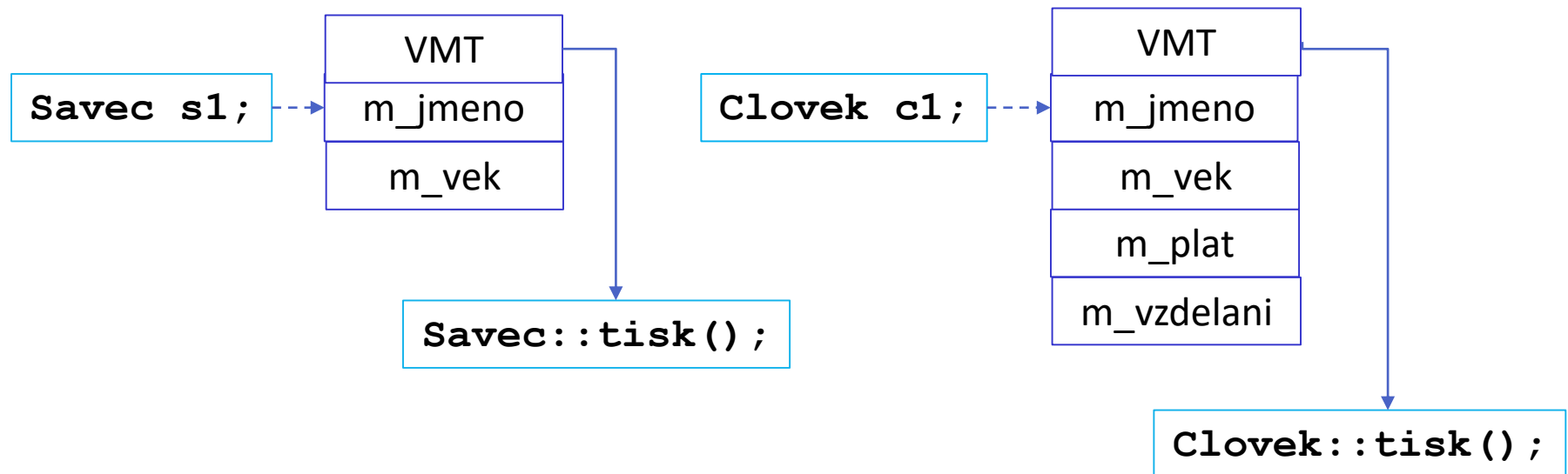
class Potomek: public Rodic {
    int p;
public:
    Potomek(int a):p(a) {};
    ~Potomek() {cout<<"\nmazani potomka";}
    void tisk() {cout<<"\npotomek
hodnota:"<<p;}
};
```

```
int main() {
    Rodic * pole[2];
    pole[0] = new Rodic(10);
    pole[1] = new Potomek(100);
    pole[0]->tisk();
    pole[1]->tisk();
    delete pole[0];
    delete pole[1];
    system("PAUSE");
    return 0;
}
```

```
rodic - hodnota:10
potomek hodnota:100
mazani rodice
mazani potomka
mazani rodicePokračujte st
```

Virtuální funkce – tabulka funkcí

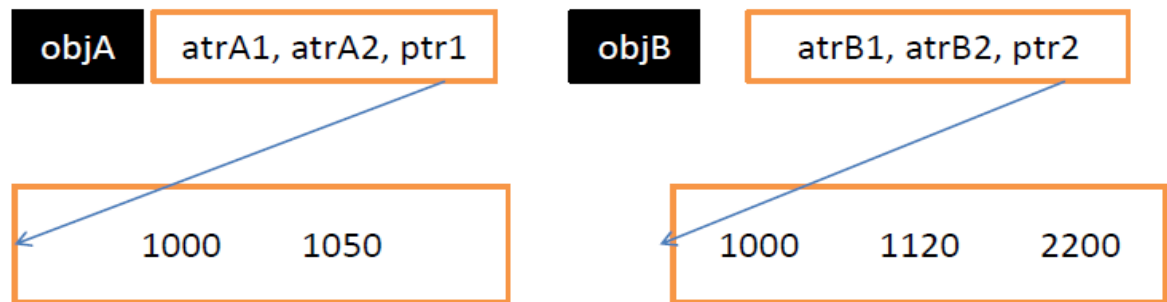
- Statická vazba zná adresu volání členské metody již v době překladu.
- Dynamická vazba využívá tabulku virtuálních metod – zde za běhu programu najde adresu členské metody.
- Každý objekt má takovou tabulku – přístupná je přes skrytý ukazatel.
- Tabulka obsahuje adresy virtuálních funkcí definované ve třídě a adresy funkcí, které překrývají virtuální funkce.



Virtuální funkce – tabulka funkcí

- Statická vazba zná adresu volání členské metody již v době překladu.
- Dynamická vazba využívá tabulku virtuálních metod – zde za běhu programu najde adresu členské metody.
- Každý objekt má takovou tabulku – přístupná je přes skrytý ukazatel.
- Tabulka obsahuje adresy virtuálních funkcí definované ve třídě a adresy funkcí, které překrývají virtuální funkce.

```
class A {  
public:  
    virtual void a1() {};  
    virtual void a2() {};  
}  
class B : public A {  
public:  
    void a2()  
    virtual void b1{};  
}  
A objA;  
B objB;
```



Virtuální funkce

- Konstruktor nemůže být virtuální – není děděn, lze ho použít jen v konstruktorech.
- Destruktor by měl být vždy virtuální – uvolnění případné dynamické paměti správného objektu.
- Spřátelená funkce nemůže být virtuální – není členskou metodou.
- Není-li metoda překryta v potomkovi, je použita virtuální metoda základní třídy.

```
class Savec {  
private:  
    string jmeno;  
    int vek;  
...};  
class Clovek: public Savec {  
private:  
    string prijmeni;  
...}
```

Použití

```
Savec s("Zak",10);
Clovek c("Ivan",30,"Hrozny");

Savec temps = s;          // lze - pokud dynamická data -> kopírující
                             konstruktor
Clovek tempc = c;          // lze - využít kopírující konstruktor rodiče a
                             // dodělat kopírování pro vlastní dyn data
Savec tempsavec = c;       // lze - pokud dynamická data -> kopírující
                             // konstruktor
Clovek tempclovek = s;     //nelze -> konverzní funkce = konstruktor s 1
                             // argumentem

Savec sa1; Clovek cl1;
sa1 = s;                   // lze - pokud dynamická data -> operátor přiřazení
cl1 = c;                   // lze - pokud dynamická data - využít operátor
                             // přiřazení základní třídy a přiřadit vlastní data
                             // v operátoru přiřazení
sa1 = c;                   // lze - pokud dynamická data -> operátor přiřazení
cl1 = s;                   // nelze -> konverzní funkce
```


Kopírování a přiřazení v dědičnosti

```
class Savec {
private:
    char *jmeno;
    int vek;
public:
    ~Savec() {delete [] jmeno;cout<<"delS";};
    Savec(){jmeno = 0; vek = 0;}
    Savec(char * jm, int ve):vek(ve){
        jmeno = new char [strlen(jm)+1];
        strcpy(jmeno,jm);
    }
    void tisk() {cout<<"\n"<<jmeno<<" " <<vek;}
    Savec(const Savec & s) {
        jmeno = new char [strlen(s.jmeno)+1];
        strcpy(jmeno,s.jmeno);
        vek = s.vek;
    }
    Savec & operator=(const Savec & s) {
        if (this ==&s) return *this;
        delete [] jmeno;
        jmeno = new char [strlen(s.jmeno)+1];
        strcpy(jmeno,s.jmeno);
        vek = s.vek;
        return *this;
    }
};
```

```
class Clovek : public Savec {
private:
    char * prijmeni;
public:
    ~Clovek() {delete [] prijmeni;cout<<"delC";};
    Clovek() {prijmeni =0;};
    Clovek(char *jm, int ve, char *pr):Savec(jm,ve) {
        prijmeni = new char [strlen(pr)+1];
        strcpy(prijmeni,pr);
    }
    Clovek (const Savec &s) : Savec(s) {
        prijmeni = new char [6];
        strcpy(prijmeni,"NEDEF");
    }
    Clovek (const Clovek & c ) : Savec(c) {
        prijmeni = new char [strlen(c.prijmeni)+1];
        strcpy(prijmeni,c.prijmeni);
    }
    Clovek & operator=(const Clovek & c) {
        if (this == &c) return *this;
        Savec::operator =(c);
        delete [] prijmeni;
        prijmeni = new char [strlen(c.prijmeni)+1];
        strcpy(prijmeni,c.prijmeni);
    }
    void tisk() {
        Savec::tisk();
        cout << " pr: " << prijmeni;
    }
};
```

Zapouzdření programu

```
//osoba.h
class Osoba {
private:
    string jmeno;
    int vek;
public:
    Osoba() {}
    int getVek() {return vek;}
    string getJmeno() {return jmeno;}
    Osoba(string jm, int ve):vek(ve) {
        jmeno=jm;}
    void tisk() {
        cout<<"\n"<<jmeno<<" "<<vek;}
};
// případně osoba.cpp
```

```
// seznamosob.h
class SeznamOsob {
    Osoba *pole_osob[10];
    int pocet_osob;
public:
    SeznamOsob() { pocet_osob = 0; }
    int Pridej_osobu();
    void tisk();
    ~SeznamOsob();
};
```

Zapouzdření programu (seznam osob)

```
// seznamosob.cpp
int SeznamOsob::Pridej_osobu() {
    string jm;
    int ve;
    cout << "\nZadej jmeno: "; cin >> jm;
    if (jm == "konec") return 1;
    cout << "\nZadej vek: "; cin >> ve;
    pole_osob[pocet_osob++] = new Osoba(jm, ve);
    return 0;
}

void SeznamOsob::tisk() {
    for (int i = 0; i < pocet_osob; i++) {
        cout << "\n os jmeno: " <<
            pole_osob[i]-> getJmeno() <<
            "\n vek: " <<
            pole_osob[i]-> getVek();
    }
}

SeznamOsob::~SeznamOsob() {
    for (int i = 0; i < pocet_osob; i++)
        delete pole_osob[i];
}
```

```
// main.cpp

#include "seznamosob.h"

int main() {
    SeznamOsob osoby;
    while (1) {
        if (osoby.Pridej_osobu())
            break;
    };
    osoby.tisk();
    return 0;
}
```

Abstraktní třídy

- Generalizace – dalším nástrojem pro její dosažení jsou AT.
- AT – vytváří rozhraní, pomocí kterého je řízeno chování podobných objektů.
- Hledají se společné vlastnosti různých objektů.
- Hledají se metody – rozhraní pro práci s nimi:
 - např. zobrazení objektů na cílovou plochu (zobrazit),
 - není podstatné, jaká přesně bude (obrazovka, tiskárna) = AT ZobrazovaciZarizeni,
 - např. tiskárny = AT Tiskarna,
 - není podstatná technologie tisku (tisk).
- Rozšiřuje princip dědičnosti – AT je základní třídou.

Abstraktní třídy (pokr.)

- AT – hlavní motivace je tvorba rozhraní, jehož implementace se upřesňuje dle typu objektu.
- Příklad : všechny grafické objekty je možné vykreslit.
- Implementace společného předka AT – GeoUtvár.
- Do rozhraní AT zahrneme metody, které mají příslušníci třídy obecně společné, ale rozdílně implementované.
- Veřejné metody vytvoří rozhraní AT, např. tisk – všechny musí jít zobrazit, ale každý rozdílně.
- Potomci AT implementují toto chování.
- AT nemá důvod pro existenci vlastní instance.
- Protože je ale použito principu dědičnosti je vždy při vytváření potomka volán konstruktor rodiče.

Abstraktní třídy (pokr.)

- Pro vytvoření AT je nutné implementovat minimálně jednu čistě virtuální metodu.
- Ta má v AT pouze svoji deklaraci, implementace je v potomcích.
- Předpis:
`virtual navrat_typ metoda (argumenty) = 0;`
- Potomek překrývá tyto virtuální metody = dynamická vazba.
- AT nevytváří svoje instance – nelze z ní vytvořit objekt.
- AT – může obsahovat implementace některých metod.
- AT – může obsahovat i společná data.
- Pokud jsou všechny metody čistě virtuální = čistě abstraktní třída.
- Čistě virtuální metoda umožní definovat u předka metodu bez implementace a tím vynutí její implementaci v potomcích.

Abstraktní třída GeoUtvár

- Pro konstrukci některých geometrických útvarů lze použít vždy souřadnice středu a vzdálenost k hranicím. U všech lze vypočítat obsah.

```
class GeoUtvár {
private:
    int m_x;
    int m_y;
    static int poc;
public:
    GeoUtvár(int,int);
    virtual double obsah() = 0;
    ~GeoUtvár();
    friend float vzdalenost(GeoUtvár * g1,
                           GeoUtvár * g2) {
        return sqrt(
            (pow((float)g2->m_x-g1->m_x,2)+
             pow((float)g2->m_y-g1->m_y,2)
            ));
    }
};
```

```
class Ctverec:public GeoUtvár {
private:
    int m_vzd;
public:
    double obsah();
    Ctverec(int, int, int);
};

class Obdelnik:public GeoUtvár {
private:
    int m_vzd1;
    int m_vzd2;
public:
    double obsah();
    Obdelnik(int, int, int, int);
};
```

Abstraktní třída GeoUtvár (pokr.)

- Pro konstrukci některých geometrických útvarů lze použít vždy souřadnice středu a vzdálenost k hranicím. U všech lze vypočítat obsah.

```
int GeoUtvár::poc = 0; // statická proměnná
```

```
GeoUtvár::GeoUtvár(int a, int b) {  
    m_x = a;  
    m_y = b;  
    poc++;  
};
```

```
Ctverec::Ctverec(int a, int b, int c)  
    :GeoUtvár(a,b) {  
    m_vzd = c;  
};
```

```
GeoUtvár::~~GeoUtvár() {  
    poc --;  
    cout<<"\n pocet obj : "<<poc;  
};
```

```
Obdelnik::Obdelnik(int a,int b,int c,int d):  
GeoUtvár(a,b) {  
    m_vzd1 = c;  
    m_vzd2 = d;  
};
```

```
double Ctverec::obsah() {  
    return (4*m_vzd*m_vzd);  
};
```

```
double Obdelnik::obsah() {  
    return (2*m_vzd1*2*m_vzd2);  
};
```


Abstraktní třída GeoUtvár (pokr.)

```
int main(int argc, char* argv[]) {
    GeoUtvár * uk[2];
    uk[0] = new Ctverec(2,2,2);
    uk[1] = new Obdelnik(3,3,2,3);
    for(int i = 0; i<2; i++) {
        cout<<"\n Obsah geoutvaru je: "<<uk[i]->obsah();
    }
    cout<< "\n vzdalenost: "<<vzdalenost(uk[0],uk[1]);
    for(int i = 0; i<2; i++) {
        delete uk[i];
    }
    return 0;
}
```

Čistě abstraktní třída

- Čistě abstraktní třída nemusí mít členská data, jen metody.
- Nemusí mít konstruktor.
- Musí ale mít minimálně jednu čistě virtuální metodu.

```
class GeometrickyUtvar {
public:
    virtual float obvod() = 0;
};
class Kruh : public GeometrickyUtvar {
private:
    float r;
public:
    Kruh(float polomer):r(polomer){};
    float obvod(); //lze i virtual, pokud chceme dědit další třídu
};
class Obdelnik : public GeometrickyUtvar {
private:
    float a,b;
public:
    Obdelnik(float s1, float s2):a(s1),b(s2){};
    float obvod();
};
```

Čistě abstraktní třída (pokr.)

```
float Kruh::obvod() { return 2*3.14*r; }
float Obdelnik::obvod() { return 2*(a+b); }

int main(void) {
    GeometrickyUtvor *k = new Kruh(1);
    GeometrickyUtvor *o = new Obdelnik(2,10);

    cout<< " obvod je "<< k->obvod();
    cout<< " obvod je "<< o->obvod();
    delete k;
    delete o;

    return 0;
}
```

The End